

In [1]: *# Importing and Configuration*

```
import pandas as pd
import json
import ipaddress
from tqdm.notebook import tqdm # Used for progress bars during enrichment

# Configuring pandas to display all columns in output
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

print("Libraries imported and display settings configured.")
```

Libraries imported and display settings configured.

In [2]: *# Configuration - Defining IP Networks*

```
PRIVATE_IP_CIDRS = [
    ipaddress.ip_network('10.0.0.0/8'),
    ipaddress.ip_network('172.16.0.0/12'),
    ipaddress.ip_network('192.168.0.0/16')
]

IP_COMPANY_MAPPING = {
    'Microsoft': [
        ipaddress.ip_network('20.0.0.0/14'),
        ipaddress.ip_network('40.74.0.0/15'),
        ipaddress.ip_network('13.107.6.0/24'),
    ],
    'Google': [
        ipaddress.ip_network('8.8.8.8/32'),
        ipaddress.ip_network('172.253.0.0/16'),
        ipaddress.ip_network('35.192.0.0/12'),
    ],
    'Amazon': [
        ipaddress.ip_network('52.94.20.0/24'),
        ipaddress.ip_network('3.232.0.0/14'),
        ipaddress.ip_network('18.244.0.0/15'),
    ],
    'Facebook': [
        ipaddress.ip_network('157.240.0.0/16'),
        ipaddress.ip_network('69.63.176.0/20'),
    ]
}

print("IP Network ranges (Internal and Company-specific) defined.")
```

IP Network ranges (Internal and Company-specific) defined.

In [3]: *# Helper Functions*

```
def check_ip_internal(ip_address_str):
    """Determines if an IP is internal (1) or external (0)."""
    try:
        ip_addr = ipaddress.ip_address(ip_address_str)
```

```

        for net in PRIVATE_IP_CIDRS:
            if ip_addr in net:
                return 1
        return 0
    except ValueError:
        return 0

def determine_company_asn(ip_address_str):
    """Maps a public IP to a company name or 'None'."""
    # Skipping internal IPs from checking against the public company list
    if check_ip_internal(ip_address_str) == 1:
        return 'None'

    try:
        ip_addr = ipaddress.ip_address(ip_address_str)
        for company_name, network_list in IP_COMPANY_MAPPING.items():
            for network in network_list:
                if ip_addr in network:
                    return company_name
        return 'None' # Everything else is 'None'
    except ValueError:
        return 'None'

print("Enrichment helper functions defined.")

```

Enrichment helper functions defined.

In [4]: *# Loading Data and Executing Enrichment*

```

log_file_name = 'nflog.json'
try:
    # Loading the JSON Log file
    netflow_df = pd.read_json(log_file_name, lines=True)
    print(f" Successfully loaded {len(netflow_df)} records from {log_file_name}.")
except FileNotFoundError:
    print(f" Error: The file '{log_file_name}' was not found.")
    raise

print("\n Starting Log Enrichment (this may take a moment)...")

# 1. Enriching src_ip_internal
tqdm.pandas(desc="Enriching src_ip_internal")
netflow_df['src_ip_internal'] = netflow_df['src_ip'].progress_apply(check_ip_intern

# 2. Enriching des_ip_internal
tqdm.pandas(desc="Enriching des_ip_internal")
netflow_df['des_ip_internal'] = netflow_df['dest_ip'].progress_apply(check_ip_inter

# 3. Enriching des_ip_company
tqdm.pandas(desc="Enriching des_ip_company")
netflow_df['des_ip_company'] = netflow_df['dest_ip'].progress_apply(determine_compa

print("\n Log Enrichment Complete.")

```

Successfully loaded 42766 records from nflog.json.

Starting Log Enrichment (this may take a moment)...

```

Enriching src_ip_internal: 0%|          | 0/42766 [00:00<?, ?it/s]
Enriching des_ip_internal: 0%|          | 0/42766 [00:00<?, ?it/s]
Enriching des_ip_company: 0%|          | 0/42766 [00:00<?, ?it/s]
Log Enrichment Complete.

```

In [5]: *# Verification and Export*

```

print("--- Verification of Enriched Data ---")

# Displaying a preview of the enriched data with the new columns
print("\nFirst 10 rows showing new columns:")
print(netflow_df[['src_ip', 'dest_ip', 'src_ip_internal', 'des_ip_internal', 'des_i

# Displaying the value counts for the key categorical column
print("\nValue counts for des_ip_company (ASN categorization):")
print(netflow_df['des_ip_company'].value_counts())

# Saving the enriched log file
netflow_df.to_json('enriched_netflow_log.json', orient='records', lines=True)
print("\n Enriched data saved to 'enriched_netflow_log.json'.")

```

--- Verification of Enriched Data ---

First 10 rows showing new columns:

	src_ip	dest_ip	src_ip_internal	des_ip_internal	des_ip_company
0	10.5.55.1	239.255.255.250	1	0	None
1	10.0.1.57	192.168.1.34	1	1	None
2	10.0.1.99	192.168.1.130	1	1	None
3	172.16.0.2	192.168.0.50	1	1	None
4	172.16.0.2	65.55.44.109	1	0	None
5	192.168.163.136	65.55.44.109	1	0	None
6	10.0.1.31	192.168.1.130	1	1	None
7	10.0.1.64	192.168.1.130	1	1	None
8	192.168.1.51	10.5.55.20	1	1	None
9	192.168.1.51	10.5.55.20	1	1	None

Value counts for des_ip_company (ASN categorization):

```

des_ip_company
None      42541
Google     225
Name: count, dtype: int64

```

Enriched data saved to 'enriched_netflow_log.json'.

In []: